

Research note

Project: Spoke (LLD practice platform)

Author: Shubhangini

Repo: <https://github.com/shubhangini67/spoke-lld-practice>

Live: <https://spoke-lld-practice.vercel.app>

CipherSchools 2-day assignment. Written before I locked the studio format.

What goes wrong when I practice LLD

I can usually get boxes on a page for Parking Lot or Elevator. I cannot tell if they are any good.

Then I do one of three things: open a GitHub repo and rename my classes to match, paste the sketch into ChatGPT and get a lecture about Strategy, or stop.

The gap is not “I don’t know what a class diagram is”. It is that there is no simple way to submit a design, get feedback that still allows other valid designs, keep the last attempt, and actually face the follow-up an interviewer will ask.

What I looked at

I did not do a full market survey. I looked at the stuff people actually open.

Hello Interview’s LLD guide is the most honest about what the round scores: problem analysis, class design, extensibility, talking. They push clarifying questions first and a follow-up at the end. Almost no practice tool does both.

Grokking / Educative is good for a first pass. Weak as a loop because the feedback is “here is our solution”.

LLD Mastery / AlgoMaster-style playgrounds are closer: clarify, diagram, code, AI comments. They are big products. They still steer you toward a canonical shape, and they don’t really freeze *your* attempt as something you can diff later.

Codezym and other machine-coding IDEs are LeetCode for LLD. Hidden tests, real code. Fine for implementation under a timer. Bad for design quality: a god class that parks cars still parks cars, and a valid design with different method names fails.

GitHub dumps (BookMyShow, Logger, Parking Lot) are useful after you already know what you are trying to say. Harmful as the first move, because one naming becomes “correct”.

Raw ChatGPT is fast and inconsistent. No problem-specific checklist, no history, and it loves “add a Strategy” without asking if that behaviour even belongs on the type you have.

Human mocks are still the best for “can you defend this”. I cannot book one at 11pm. I wanted a rehearsal for that conversation, not a replacement.

What I think a real attempt has to contain

Interviewers don't score a screenshot. They score a conversation.

1. Scope you actually locked. One floor or many? Coins only, or cards later? If this is missing you designed a different system than the one in the room.
2. Named types with a one-line responsibility. Smallest thing someone can point at.
3. Who talks to whom. A list of nouns is just vocabulary.
4. One walked-through use case. “User parks a truck” across objects, in order. Candidates skip this. Interviewers ask for it.
5. Something you rejected, and why. That is the senior-ish signal.

Code can show a seam. For this MVP it should not be the unit of practice. Tests will pass a mess.

Feedback when more than one design is valid

Score capabilities (what the design must support) and seams (where Monday's requirement lands). Show evidence so I can argue with the checker. Alternatives as “this shape fits when...”, not *the* answer. After the first review the follow-up is the other half of the round. If you don't practice that, you practiced half an interview.

Do not freeze a class list. A `Stall` that assigns vehicles still covers spot assignment. I put a test on that so I wouldn't quietly regress into a golden-name checker.

Deterministic vs LLM

Coverage, graph smells, walkthrough, scope, submit validation, and the state machine are deterministic. They have to work with no key, and I have to be able to explain them.

Qualitative stuff (defense, other shapes, interviewer questions) is nicer with a model, on top of the checker, never instead of it. The model does not own the score. If it names a type I didn't write, that line gets dropped.

What I tried to close in two days

Most tools skip clarifying questions, skip the narrated use case, skip the follow-up, skip revision history. Spoke is small and stubborn about those four.

Out of scope: accounts, contest timers, a drawing canvas, hidden JUnit, a message bus for evaluation.

MVP I locked after this: five problems, one structured design (not a Java IDE), hybrid evaluation, attempt history with clone-on-revise.